

PTU DC presents

# Pattern Coding

A comprehensive pdf series with 5 pattern-based  
questions in each edition

**DC24**  
Edition #1/5

# ACKNOWLEDGEMENT

As I present the first edition of "Pattern Coding" on this auspicious occasion of World Design Day, my heart is filled with gratitude and appreciation for those who have contributed to the realization of this mini-PDF series. This project was initiated with the aim to provide a structured learning pathway in coding for our juniors and peers at PTU, and its fruition is a testament to the collaborative spirit of our academic community.

Firstly, I would like to express my profound thanks to Dr.K Sathiyamurthy, Professor CSE and the Convenor of PTU Design Club, whose guidance has been indispensable. His insights and unwavering support have not only shaped this edition but have also inspired me to push the boundaries of what we can achieve through educational initiatives. His dedication to fostering a learning environment is truly commendable. I thank Dr. G. Ramya, coding trainer,TNP PTU for supporting me with all necessary content and guidance while in need.

My sincere thanks also go to Yomi, Krishna, Kuhan, and Anupama, whose efforts in organizing and compiling the content have been vital to this series' success. And my special thanks to Rupam Chirom, the founder of this club for his constant support and brotherly guidance.

As the former president of the Design Club at PTU, I have witnessed firsthand the vibrant community we have built together. The intention of this series is to instill the idea that coding more of pattern-based questions can help us simplify any complex question at ease.

This series would not have been possible without the cumulative effort and enthusiasm of all those involved. I hope that "Pattern Coding" serves as a valuable resource for many and that it encourages our students to explore the fascinating world of coding with curiosity and passion.

It is also important to note that while this material begins at a beginner-friendly level, it is designed to increase in complexity with each subsequent edition, encouraging continual learning and growth.

Lastly, I extend my heartfelt thanks to all the readers and future contributors who will take this series forward. May this journey of learning and discovery continue to grow and inspire.

With love and much gratitude,  
Gokulnath Pugazhendi  
B.Tech Final Year, CSE  
PTU

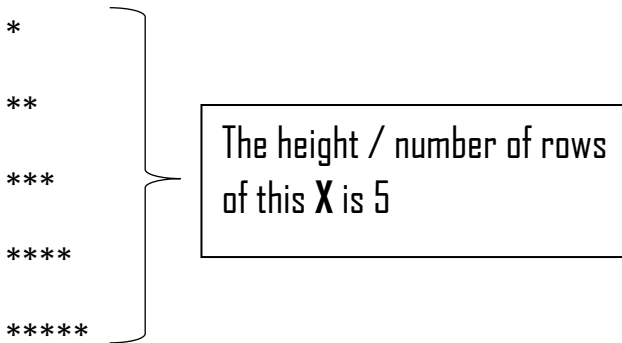
# INDEX

|     |                      |    |
|-----|----------------------|----|
|     | ACKNOWLEDGEMENT..... | 2  |
| 1.1 | PROBLEM 1.....       | 5  |
| 1.2 | PROBLEM 2.....       | 9  |
| 1.3 | PROBLEM 3.....       | 14 |
| 1.4 | PROBLEM 4.....       | 18 |
| 1.5 | PROBLEM 5.....       | 24 |
|     | DISCLAIMER.....      | 28 |



- l) Write program to print pattern of stars, with each row containing an increasing number of stars up to a specified number of rows set by the user.

### APPROACH:



First, let's get the row height as input from the user.

```
cin>>rows;
```

The outer loop iterates through each row of the pattern, starting from the first row and continuing up to the total number of rows specified by the user. The loop variable `i` ranges from 1 to the value of `rows`, inclusive.

```
for (int i = 1; i <= rows; i++)
```

The value of 'i' increments until the condition  $i \leq \text{rows}$  is true

The inner loop prints stars for the current row, with the loop variable  $j$  ranging from 1 to  $i$ , inclusive. In each iteration of the inner loop, a star is printed, and the number of stars printed increases by one with each iteration of the outer loop.

```
for (int j = 1; j <= i; j++) {  
    cout << " *";  
}
```

If we are in first row ( $i=1$ ) this inner loop prints maximum of  $i$  stars (here 1).

In the first row, ' $i = 1$ ', it prints one star.

\*

In the second row, ' $i = 2$ ', it prints two stars and so on.

\* \*

At last, after printing the stars for each row, we use '`cout<<endl;`' to move to the next line for the next row.

## **CODE:**

```
#include <iostream>

using namespace std;

int main() {

    int rows;

    // get row height from user

    cout << "Enter the number of rows: ";
    cin >> rows;

    // Outer loop iterates for each row of the pattern

    for (int i = 1; i <= rows; i++) {

        // Inner loop prints stars for the current row

        for (int j = 1; j <= i; j++) {

            cout << "*";

        }

        cout << endl; // Move to the next line after each row

    }

    return 0;

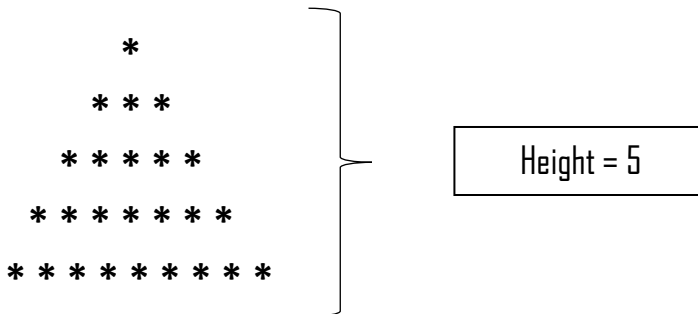
}
```





2) Write a program to print triangular star pattern

**APPROACH:**



From the above pattern, we can observe the following

| Row | Spaces | Stars |
|-----|--------|-------|
| 1   | 4      | 1     |
| 2   | 3      | 3     |
| 3   | 2      | 5     |
| 4   | 1      | 7     |
| 5   | 0      | 9     |

Let us get number of rows as the input from the user.

```
cin>>n
```

Three integer variables, i, j, k are declared to be used as loop counters in the subsequent loops.

We start a for loop (outer loop) that will iterate from '**i = 1**' to '**i = n**', that represents each row in the triangular pattern.

```
for(i = 1; i <= n; i++)
```

Value of i increments until the condition '**i <= n**' is true

Within the outer loop, we use another for loop incrementing from '**j = 1**' to '**j = n-i**', to print the necessary spaces before the stars to left-align them. We call that as nested J loop.

```
for(j = 1; j <= n-i; j++){  
    cout<<" ";  
}
```

In the first row '**i = 1**', we have the maximum number of spaces (**n-1**) to align the first star to the rightmost position.

As we progress to the next row, the number of spaces decreases by 1, creating a left alignment effect.

After printing the spaces, another for loop is initialized to print the stars. We call it nested K loop.

This loop increments from '**k = 1**' to '**k <= 2\*i-1**'.

```
for(k = 1; k <= 2*i-1; k++){  
    cout<<"* ";  
}
```

In the first row, '**i = 1**', it prints one star. {4 spaces by J loop and 1 star by K loop}

\*

In the second row, '**i = 2**', it prints three stars {3 spaces by J loop and 3 star by K loop}.

\* \* \*

At last, after printing the spaces and stars for each row, we use '**cout<<endl;**' to move to the next line for the next row, creating the vertical spacing between rows.

Thus creating the triangular star pattern.

## **CODE:**

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of rows: ";
    cin >> n;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n - i; j++) {
            cout << " ";
        }
        for (int k = 1; k <= 2 * i - 1; k++) {
            cout << "*";
        }
        cout << endl;
    }
    return 0;
}
```

0, 1, 1, 2, 3,  
5, 8, 13, 21,  
34, 55, 89,  
144, 233,  
377, 610,  
987...

3) Noticed the pattern before? Yes, it is indeed the Fibonacci Sequence.  
Let's approach this basic problem systematically

### **APPROACH:**

Fibonacci sequence starts with a 0 and followed by a 1. Then it is the sum of the two numbers before that follows the sequence.

Let's explore in detail.

First number: 0 (\*default)

Second number: 1 (\*default)

Third number:  $0+1=1$

Now the sequence is 0 1 1

Fourth number is  $1+1=2$  since it is the addition of previous two numbers.

Fifth number:  $2+1=3$

Therefore, the sequence continues to go on as

0 1 1 2 3 5 8 13 21 34 55 89 ...

But the core logic is as simple as the sum of the last two numbers.  
To keep track of that, we can use temporary variables to store the numbers.

Let's get the number of terms to be printed in n

```
cin >> n
```

Store 0 and 1 in two separate variables t1 and t2 so as to keep track of the numbers while printing the sequence

```
int t1 = 0, t2 = 1
```

Now, create a for loop with i as the iterator that prints the sequence.

If i is 1, print t1 which is 0. If i is 2, print t2.

```
if(i == 1){  
    cout<< t1;  
    continue;  
}
```

The continue keyword is used when a particular condition is met and you want the for loop to go for the next iteration rather than executing the code beneath it. It is the opposite of break.

We'll then have a variable to store the sum of the two numbers as in this case, it's t1 + t2 followed by swapping the numbers so that the previous 2 numbers are replaced with the new ones.

```
nextTerm = t1 + t2;  
t1 = t2;  
t2 = nextTerm;
```

Finally print the nextTerm to continue the sequence.

## CODE:

```
#include <iostream>

using namespace std;

int main() {

    int n, t1 = 0, t2 = 1, nextTerm = 0;

    cout << "Enter the number of terms: ";

    cin >> n;

    cout << "Fibonacci Series: ";

    for (int i = 1; i <= n; ++i) {

        // Prints the first two terms.

        if(i == 1) {

            cout << t1 << " ";

            continue;

        }

        if(i == 2) {

            cout << t2 << " ";

            continue;

        }

        nextTerm = t1 + t2;

        t1 = t2;

        t2 = nextTerm;

        cout << nextTerm << " ";

    }

    return 0;

}
```



1

1 1

1 2 1

1 3 3 1

1 4 6 4 1

1 5 10 10 5 1

4) The pattern you see is famously known as the Pascal's Triangle. Let's dive deep into the structure

### **APPROACH:**

The code uses a two-dimensional array to store and print Pascal's Triangle up to a given number of rows, specified by the user. This explanation will break down each part of the code for clarity.

```
### Includes and Namespace
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

- `#include <bits/stdc++.h>`: This line includes all standard C++ libraries. This is typically used in programming contests for convenience but is not recommended for production code due to potential compile-time overhead and portability issues.

- `using namespace std;`: This allows you to use standard library objects and classes (like `cout` and `cin`) without the `std::` prefix.

### Function to Print Pascal's Triangle

```
void printPascal(int n)
```

```
{
```

```
    int arr[n][n];
```

- `void printPascal(int n)`: This function generates and prints the first `n` rows of Pascal's Triangle.

- `int arr[n][n]`: This declares a 2D array of size `n x n`. Each element of the array will store an integer from Pascal's Triangle.

### Looping to Generate Pascal's Triangle

```
for (int line = 0; line < n; line++)
```

```
{
```

```
    for (int i = 0; i <= line; i++)
```

```
{
```

- The outer loop (`line`) iterates over each row from `0` to `n-1`.

- The inner loop (`i`) iterates from `0` to the current row number (`line`). The reason for this is that the number of elements in each row of Pascal's Triangle is equal to the row number plus one.

### ### Conditionals to Fill the Triangle

```
if (line == i || i == 0)
    arr[line][i] = 1;
else
    arr[line][i] = arr[line - 1][i - 1] + arr[line - 1][i];
cout << arr[line][i] << " ";
```

- `if (line == i || i == 0)`: This checks if the current position is at the start or end of a row. The first and the last elements of every row in Pascal's Triangle are always `1`.

- `else`: This calculates the value of the current element as the sum of the element directly above it and the element to the left of the element directly above. This utilizes the property of Pascal's Triangle where each element is the sum of the two directly above it.

- `cout << arr[line][i] << " "`: This prints the current element followed by a space.

### Newline After Each Row

```
cout << "\n";
```

- After completing each row, a newline character is printed to start a new row.

```
### Main Function
```

```
int main()  
{  
    int n = 5;  
    printPascal(n);  
    return 0;  
}
```

- `int n = 5;`: Specifies the number of rows of Pascal's Triangle to print.

- `printPascal(n);`: Calls the function to print Pascal's Triangle with `n` rows.

- `return 0;`: Standard for the main function, indicating that the program executed successfully.

The approach is intuitive, making it suitable for beginners to understand core concepts in C++ programming.

## CODE:

```
#include <bits/stdc++.h>

using namespace std;

void printPascal(int n)
{
    int arr[n][n];

    for (int line = 0; line < n; line++)
    {
        for (int i = 0; i <= line; i++)
        {
            // First and last values in every row are 1
            if (line == i || i == 0)
                arr[line][i] = 1;
            else
                arr[line][i] = arr[line-1][i-1] + arr[line-1][i];

            cout << arr[line][i] << " ";
        }

        cout << "\n";    }
}

int main()
{
    int n = 5;

    printPascal(n);

    return 0;
}
```



Let's replicate it's basic structure, shall we?

5) Write a program to print the Cross Sign ( X ) Star Pattern.

### APPROACH:

\* \*  
\* \*  
\*  
\* \*  
\* \*

The height / number of  
rows of this **X** is 5

First, let's get the row height as input from the user.

```
cin>>row_size
```

Three integers variables, **in**, **out** and **p** are declared to be used as loop counters in the subsequent loops

The outer for loop is used to control the number of rows in the pattern

```
1 <= out <= row_size
```

from row 1 to row **row\_size**



The inner loop is to control the number of characters in each row

`l <= in <= row_size`

\* \*

\* \*

\* \*

\* \*

\* \*

When **in == out**, the star is printed i.e. in this case, when **in = 3** and **out = 3** a star is printed

With this condition, the complete left to right line in the **X** will be printed

\*

\*

\*

\*

\*

If you notice the logic behind printing the other stars, we notice that **in+out == row\_size+1** i.e. in 1<sup>st</sup> row, the star needs to be printed at the 5<sup>th</sup> position. **in** in this case is 1 and out is 5.

So concluding from this, if  $in + out == row\_size + 1$ , then star needs to be printed which is  $5 + 1$ . This is the case for other rows too. Now fabricating both the condition into a single if statement will give us

```
if( in == out || in + out == row_size + 1 )
```

If both the condition fails, print a space " " instead. This simple logic constitutes the X.

**Tip:** Before starting to code, try to find the pattern a output structure follows and attempt to confine it to a condition or a formula of your convenience. This will avoid spending unwanted time on brute-force techniques

## **CODE:**

```
#include<iostream>

using namespace std;

int main()
{
    cout<<"Enter the row size:";

    int row_size;

    cin>>row_size;

    int in,out,p;

    for(out=1;out<=row_size;out++)
    {
        for(in=1;in<=row_size;in++)
        {
            if(in==out || in+out==row_size+1)
        {
            cout<<"*";

        }
        else
        {
            cout<<" ";

        }
        }
        cout<<"\n";

    }
}
```

## DISCLAIMER

This work, titled "Pattern Coding," is the intellectual property of the Design Club at PTU. Replication and distribution of this PDF series are permitted provided that due credit is given to the Design Club of PTU as the original source. This work is intended solely for educational purposes and is not to be commercialized in any form. It has been created with the intention of benefiting the student community, and any use of this material outside of its educational scope should strictly adhere to these guidelines to uphold the spirit in which it was created.

